Introduktion till C++

Deluppgift 1.2: Fler spel

Uppgiftens målsättning

Att göra den studerande bekväm med grundläggande funktionalitet i språket som funktioner, variabler och flödeskontroll. Studeranden kommer även efter uppgiften att ha fått en förståelse för varför kod bör struktureras upp samt ha fått grundläggande träning i praktisk problemlösning i C++.

Uppgift

Vi ska nu ta det enkla spelet som skapades i *Uppgift 1.1* och expandera det med fler koncept. Ett av de stora koncepten vi introducerar är **pengar** och möjligheten att **förlora spelet** om pengarna tar slut. Vi introducerar även **ett spel till** och en **huvudmeny** för att välja vilket spel du vill spela. När vi nu har flera spelbord så lägger vi även till en meny för i respektive spelbord.

Features

Huvudmeny
En enkel meny där spelaren kan välja att gå till något av borden eller att lämna casinot.
Om hen lämnar casinot presenteras hur mycket pengar man har kvar till spelaren. Spelet skall inte stängas ned innan användaren kunnat ta del av informationen.

Spelbord
Det ska finnas två olika spelbord: ***gissa numret*** och ***udda eller jämt***.

Från huvudmenyn ska man kunna välja att gå till ett spelbord eller att avsluta spelet.

Vid varje spelbord får spelaren veta hur mycket pengar hen har. Därefter ges ett stämningsfullt meddelande varpå regler förklaras.

Så länge spelaren uppehåller sig vid ett spelbord väljer hen från bordets meny som ger valen: *spela vid bordet*, *visa reglerna* eller *lämna bordet och återgå till huvudmeny*. Om valet är att spela vid bordet så genomförs spelet varpå spelaren återvänder till bordets meny.

Gissa numret
Kod för detta specifieras i *Uppgift 1.1*, så anpassa den koden med meny för bordet och lägg in pengar.

Udda eller jämt
Spelets regler är enkla. Spelaren satsar på udda eller jämt, sen slår man två tärningar.
Spelaren vinner om båda tärningarna visar ett udda tal och denne satsat på udda eller om båda tärningarna visar ett jämnt tal och spelaren har satsat på jämnt. Om de visar olika (dvs. ena tärningen jämnt och den andra udda) så vinner banken alltid. Totalsumman av tärningarna spelar ingen roll.

Spelare ska få välja mellan udda eller jämnt och resultaten ska presenteras per tärning.

Pengar
Spelaren ska ha tillgång till en viss mängd pengar (ni själva bestämmer hur mycket spelaren börjar med). Spelaren skall ha en summa som används för att satsas i alla spel. Följande måste vara uppfyllt:

- För varje spelrunda ska spelaren få veta hur mycket pengar hen har och frågas hur mycket pengar hen vill satsa.
- Spelaren får inte lov att satsa mer pengar än hen har utan det måste finnas felkontroller som i sådana fall instruerar spelaren att försöka igen med en korrekt summa. Spelaren får inte heller satsa en negativ summa eller 0.
- Om spelaren satsar alla sina pengar ska spelet märka detta och ge en stämningshöjande reaktion.
- Om spelaren förlorar ska de satsade pengarna försvinna.
- Om spelaren vinner ska pengarna öka med *exempelvis* 2 gånger satsningen.
- Det ska vara olika vinstmodell för de två spelen.
- Om spelarens pengar tar slut ska denna bli utslängd från rummet av säkerhetsvakterna och spelet ska ta slut.

Inlämning
Detta är en **delmomentsuppgift** som inte behöver lämnas in.
Dock **rekommenderar** vi att ni gör det för feedback på er kod samt att redan nu komma in i hur uppgifter skall lämnas in framöver.

Skapa en mapp med **uppgiftens namn**. I denna mapp skapar ni ytterligare två mappar. En som heter **Exe** och en som heter **Source**.

Source-mappen innehåller all källkod samt projekt filer.
Exe-mappen innehåller endast filer som behövs för att köra programmet.
Ta bort alla skräpfiler!!

Läs igenom ”Inlämningsinstruktioner”-dokumentet för en mer komplett genomgång om hur uppgifter skall lämnas in.

Verktyg

```
lokala variabler
globala variabler
funktioner
operatorer


assert
#include
char, short, int, long, float, double, bool, void, unsigned, enum,
enum class
if, else, else if, switch, while, do, for, return, break, continue,
const
random_device, mt19937, uniform_int_distribution
cout, cin, printf
static_cast<>
system(“cls”), system(“pause”)
```